Avdhut Parish madhavi C2 Batch
C-60 SE TU352627006

Terna Engineering College, Nerul

AIPS

Assignment No. 1

8
10

1. The recurrence relation of an algorithm is as given below.

$T(n) = T(n/2) + 1$ for $n > 1$

$= 1$ for $n = 1$

use Backward substitution method to find out the equation for the time complexity of the algorithm write the final answer in Big O notation.

→ By Backward substitution method

put $n = n/1$

$T(n) = T(n/2) + 1$ — (1)

$T(n) = T(n/4) + 1$ — [2]

Substituting eqn 2 in 1

$T(n) = T(n/4) + 1 + 1$ — [3]

similarly

$T(n/4) = T(n/8) + 1$ — [4]

subtituting eq 4 in 3

$T(n) = T(n/8) + 3$

similarly

$T(n) = T(n/16) + 4$

After K substitutions

$T(n) = T(n/2^K) + K$

$\frac{n}{2^K} = 1$

$n = 2^K$

$K = \log_2 n$

$T(n) = T(1) + \log_2 n$

$T(n) = 1 + \log_2 n$

$\therefore T(n) = O(\log n)$

2. write & analyze bubble sort algorithm with the help of example & derive its complexity for best, average & worst case

→
```
BubbleSort(A, n)
for (i = 0 , i<n-1; i++)
    for (j = 0 ; j<n-i-1 ; j++)
        if (a[j+1] < A[j])
            swap (A[j+1 ], A[j])
        end if
    end for
end for
```

In Bubble sort Outter for loop is to iterate over n - 1 number of times in an array.

The inner for loop is for comparing adjacent elements. If the left side element is bigger than right side element then swap it, using swap (A[j+1], A[j])

Example :-

A = [5, 3, 8, 4, 2]

Pass 1 :- compare Adjacent element

i. 5 > 3 then swap

result = 3, 5, 8, 4, 2

ii. 5 > 8 NO swap.

iii. 8 > 4 then swap

result = 3, 5, 4, 8, 2

iv. 8 > 2 then swap

result = 3, 5, 4, 2, 8

Pass 2:- i. 3 > 5 NO swap.

ii 5 > 4 swap

result = 3, 4, 5, 2, 8

iii. 5 > 8 NO swap

iv. ~~8 > 2~~ ~~swap~~

~~result = 3, 4, 5, 2, 8~~

Pass 3:- i. 3 > 4 NO swap.

ii. 4 > 5 NO swap

iii 5 > 2 swap :.

result = 3, 4, 2, 5, 8

iv. 5 > 8 NO swap.

Pass 4:- i. 3 > 4 NO swap

ii. 4 > 2 swap :.

result = 3, 2, 4, 5, 8

iii. 4 > 5 NO swap.

iv 5 > 8 NO swap.

Pass 5 :- i. 3 > 2 swap.

result = 2, 3, 4, 5, 8

ii. 3 > 4 NO swap

iii. 4 > 5 NO swap

iv. 5 > 8 NO swap.

Hence we got our sorted array :-

2, 3, 4, 5, 8

Terna Engineering College,Nerul

Best case $O(n)$

Average case $O(n^2)$

worst case $O(n^2)$

$T(n) = T(n-1) + (n-1)$

Expanding

$T(n) = T(n-2) + (n-2) + (n-1)$

$T(n) = T(1) + 1+2+3 \cdots (n-1)$

AS $1+2+3+\ldots+(n-1) = \frac{n(n-1)}{2}$

$\therefore T(n) = \frac{n(n-1)}{2}$

$\therefore T(n) = O(n^2)$

for worst & Average case :- $O(n^2)$

Best case $T(n) = n-1$

$\therefore T(n) = O(n)$

3. Sort the following numbers using quick sort algorithm choose the first of element of a list as pivot in each recursion. Show the entire sorting process neatly step-by-step for each iteration

43	32	22	78	63	57	91	13	47	51	19	11	37	17	69

→

Partition around 43 : we find element greater than 43 from the left & smaller than 43 from the right

pivot = 43

i = 78

j = 17

Swap.

43 32 22 17 68 57 91 13 47 51 19 11 37 78 69

pivot ↓i ↓j

Step 2 :

i = 63 (43<63)

j = 37 (43>37)

swap

43 32 22 17 37 68 57 91 13 47 51 19 11 63 78 69

pivot ↓i ↓ ↓j

Step 3 :-

i = 57 (43<57)

j = 11 (43>11)

swap

43 32 22 17 37 11 91 13 47 51 19 57 63 78,69

↓i ↓j

Step 4 :-

i = 91

j = 19

swap

43 32 22 17 37 11 19 13 47 51 91 57 63 78 69

↓i ↓ ↓j

Step 5:

i = 47

j = 13

Swap ~~as~~ i & j as didn't cross each other.

~~42 32 22 17 37 11 19 13~~

Swap j with pivot as i & j passed each other

13 32 22 17 37 11 19 43 47 51 91 57 63 78 69

So the correct first partition is

13 32 22 17 37 11 19 | 43 | 51 91 57 63 78 69

Now apply quick sort on left subarray

13 32 22 17 37 11 19

↓ ↓ ↓

Pivot i j

Step 1:

i = 32

j = 11

Swap

13 11 22 17 37 32 19

↓ ↓

i j

Step 2

i = 22

j = 11

As j crossed i we swap pivot ele. with j

∴ 11 | 13 | 22 17 37 32 19

↓

pivot

Now we apply quick sort right sub array of 2nd pass:

22 17 37 32 19

pivot i j

Step 1:

i = 37

j = 19

Swap : 22 19 19 32 37

i j

Step 2:-

i = 32

j = 19

swap pivot with j as i & j passed each other

19 17 |22| 32 37 ⟶ left sub array - (3)

Apply quick sort on left subarray (3)

19 17 ~~22~~ ~~37~~

pivot i,j

swap:-

17 , 19

∴ Sorted Sub array (left) =

11 , 13, 17, 19, 22, 32, 37

i.e

11, 13, 17, 19, 22, 32, 37 | 43 | 47, 51, 91, 57, 69, 78, 69

Right Array.

Now we apply quick sort right array.

47 51 91 57 63 78 69

pivot i j

Step 1:-

i = 51

j = 47

i.e All element are greater than pivot ∴

47 is in correct position.

Now recursively sort

51 91 57 63 78 69

pivot

Step 1:

i = 91

j = 51

51 is in correct position as all elements are greater.

Now recursively sort

91 57 63 78 69

pivot

Step 1:

i = (All elements are smaller than pivot element)

∴ 91 moves to end

57 63 78 91 | 91 |

Pivot i j

Step 1:

i = 63

j

69 57 63 78 | 91 |

Now: pivot = 69

Step 1:-

i = 78

j 63

i & j crossed ∴

63 57 | 69 | 78

Now sort 63 57

pivot = 63

57 is smaller

57, 63

∴ The right side becomes

47 51 57 63 69 78 91

Final sorted array :-

11, 13, 17, 19, 22, 32, 43, 47, 51, 57, 63, 69, 91

Q4. Explain min max algorithm & derive its complexity.

→ Algorithm.

```
Min_max (A, low, high)
  if low == high
     return (A[low], A[high])
  if high == low + 1
     if A[low] < A[high]
        return (A[low], A[high])
     else
        return (A[high], A[low])
mid = (low + high) / 2
(min1, max1) = min_max(A, low, mid)
(min2, max2) = min_max(A, mid+1, high)
return (min(min1, min2), max(max1, max2))
```

- In min_max algorithm we get the minimum element and maximum element from it by dividging the array into two halves.

- If only one element, it is both minimum & maximum
- If two element compare them & find minimum & maximum element.
- otherwise divide the array into two halves
- Recursively find minimum and maximum element of both halves.
- Combine the results of two halves, compare between the two minimum & maximum element of them & return the final minimum & maximum element.

Recurrence relation :-

$$T(n) = 2T\left(\frac{n}{2}\right) + 2$$

By master theorem

here $a = 2$, $b = 2$ $k = 0$, $p = 0$

$b^k = 2^0 = 1$

As $a > b^k$

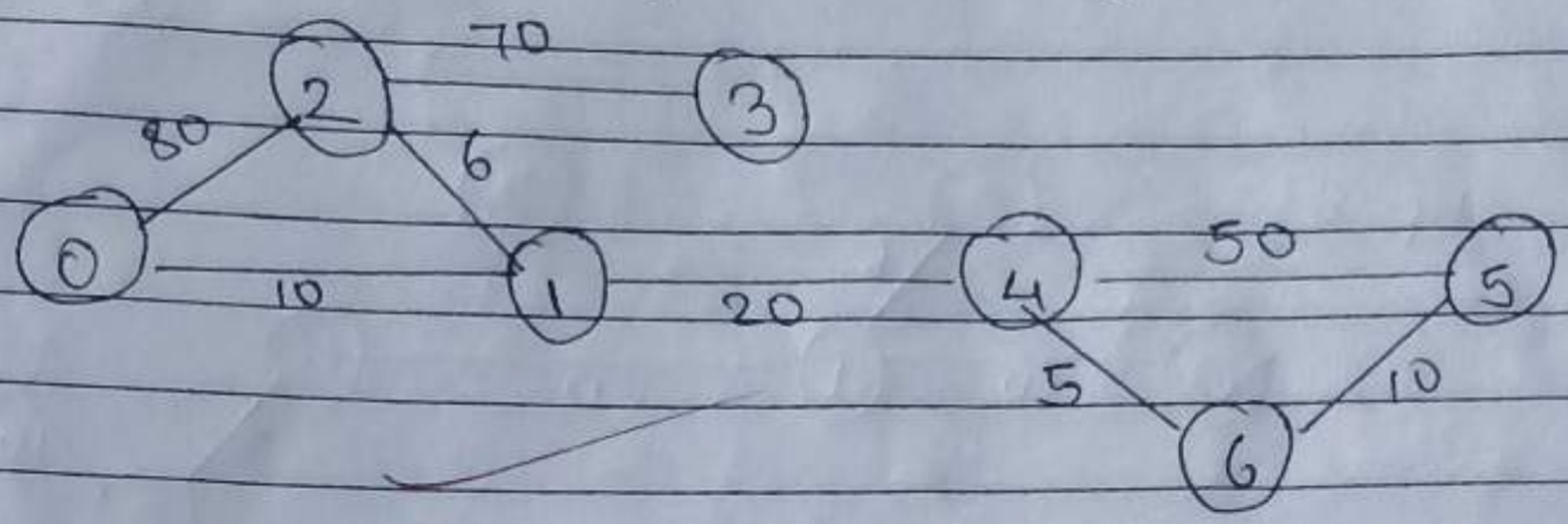
By case (1) : $T(n) = \theta(n \log_b a)$

$\therefore T(n) = \theta(n^1 \log_2 2)$

$\therefore T(n) = \theta(n)$

$\therefore$ The time complexity is $O(n)$

5. Find the minimum cost Spanning Tree (MST) for the following undirected and unweighted graph using Kruskal's algorithm. Show the entire procedure step-by-step neatly.

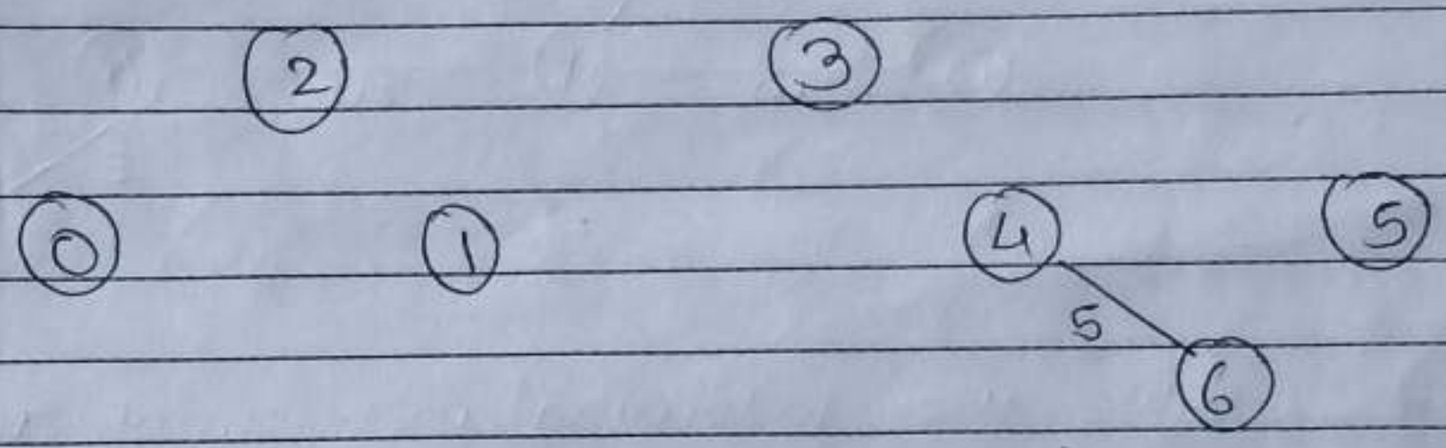


→

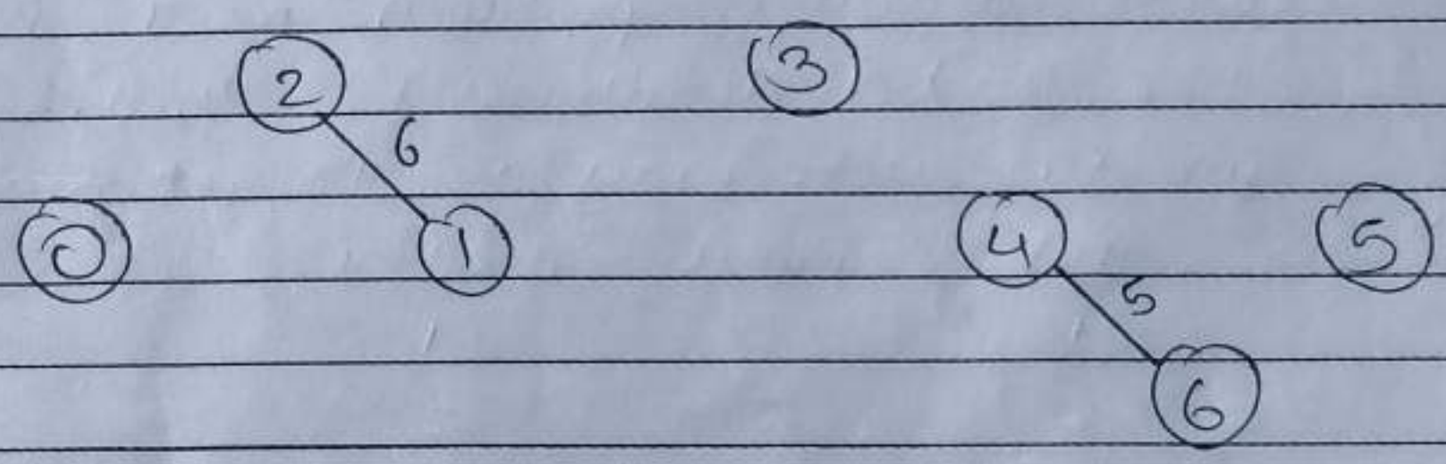
Cost = 5 + 6 + 10 + 10 + 20 + 70

= 121

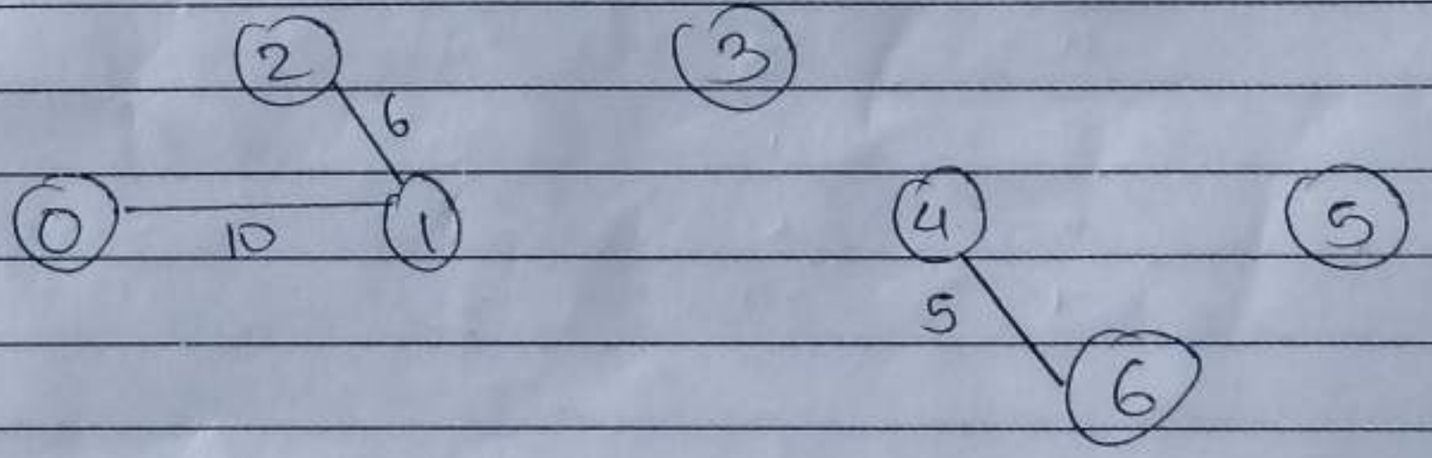
Step 1 :-



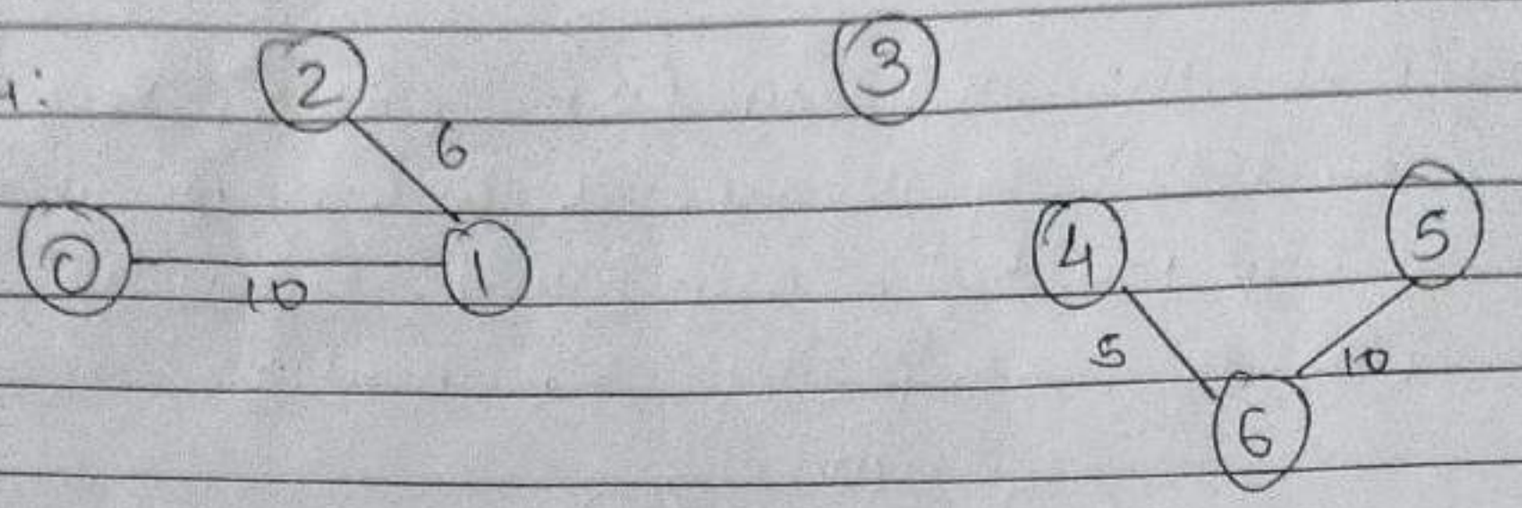
Step 2 :-



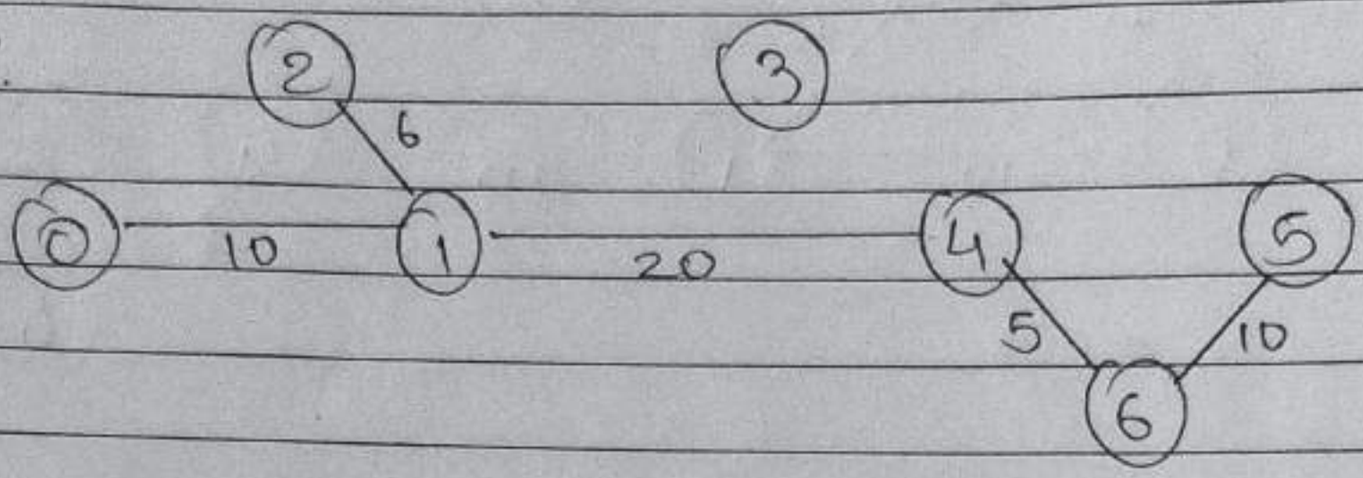
Step 3 :-



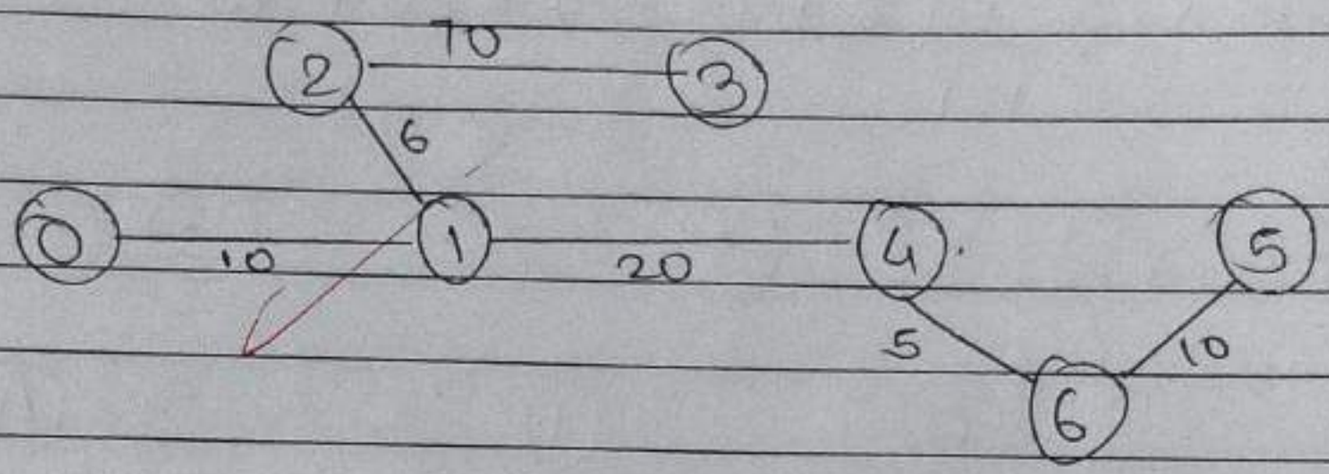
Step 4:



Step 5:



Step 6:



Q6. Solve the following instance of Job sequencing with deadlines problem n = 7, Profit = (3, 5, 20, 18, 1, 6, 30) & deadlines (1, 3, 4, 3, 2, 1, 2) schedule the job in such way as to get maximum profit.

→

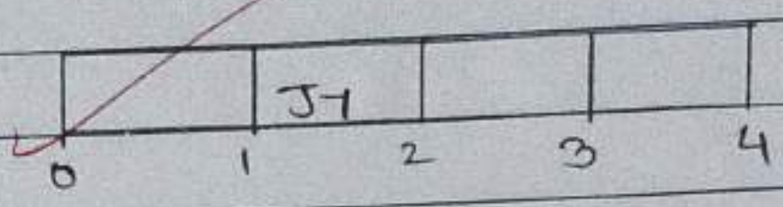
Job	Profit	deadlines
1	3	1
2	5	3
3	20	4
4	18	3
5	1	2
6	6	1
7	30	2

Arranging in decending order according to profit.

Job	Profit	deadline
7	30	2
3	20	4
4	18	3
6	6	1
2	5	3
1	3	1
5	1	2

maximum deadline = 4 ∴ we have 4 slots.

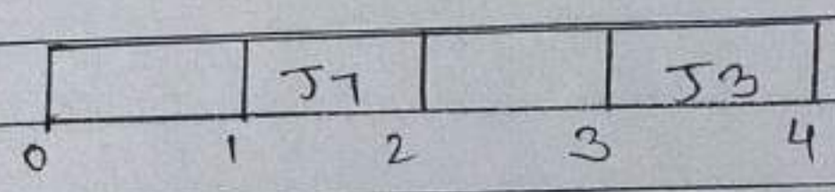
1. Job 7



Profit = 30 , deadline = 2

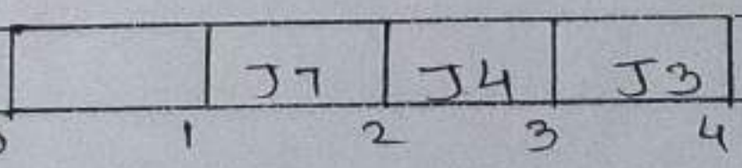
2. Job 3

Profit = 20 , deadline = 4



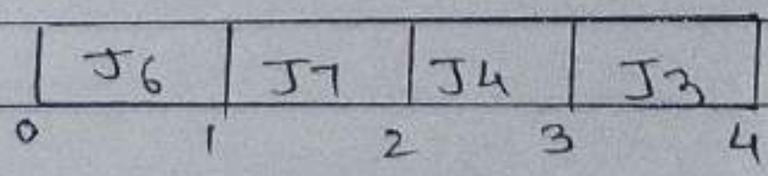
3. Job 4

Profit = 18 , deadline = 3



4. Job 6

Profit = 6 , deadline = 1



— — final order

Optimal solution: J6, J7, J4, J3

Maximum Profit = J6 + J7 + J4 + J3

6 + 30 + 18 + 20 = 74